

R1 Maksym Kotlubaiev
R2 A2406528
R3
R4
R5
R6

Learning Diary
R504TL194 Cloud Computing

1 (4)

12 February 2026

R8 **Student's written exercises**

R9

R10 Module 2 Bonus task. Testing the limits of existing sentiment models

This bonus task explores the performance and limitations of a pre-trained sentiment analysis model from the HuggingFace Hub. The goal is to apply the model to different types of textual phenomena (I choose typos, slang, and irony) and to analyze how these affect classification results. Understanding model limitations is important for real-world NLP systems where text is often noisy and context-dependent. Here is my completed practical work "[Module2BonusTask_Maksym.ipynb](#)".

When data is messy The HuggingFace blog post demonstrates a simple pipeline for sentiment analysis using transformers. It highlights that more than 200 pretrained sentiment models are publicly available and can be integrated in Python with only a few lines of code. The article explains how text is tokenized and passed through the model to produce a label and score. It also emphasizes that while these models work well on standard text, they may struggle with domain-specific language, informal spelling, or contextual nuances (Shane 2020).

Sentiment Analysis using Python

The HuggingFace blog post "Getting Started with Sentiment Analysis using Python" provides a practical introduction to applying pre-trained transformer models for sentiment classification. The article demonstrates how easily sentiment analysis can be implemented using the pipeline API, requiring only a few lines of code to load and apply a model. It highlights the availability of hundreds of publicly accessible sentiment models on the HuggingFace Hub and explains how text is tokenized, processed by a transformer architecture, and mapped to sentiment labels with associated confidence scores. While the blog emphasizes the simplicity and power of modern NLP tools, it also implicitly shows that these models rely heavily on the data they were trained on, which makes their performance sensitive to noise, domain shifts, and informal language.

Testing the sentiment analysis model

The selected model is [distilbert-base-uncased-finetuned-sst-2-english](#). It is given as an example in this task, and I thought it would be interesting to try working with it.

The model performs binary sentiment classification (positive/negative) on English text. It is lightweight and efficient but was trained primarily on clean movie review data. This model reaches an accuracy of 91.3 on the dev set (Jik 2020).

12 February 2026

With this simple code I was able to use the model:

```
classifier = pipeline(  
    "sentiment-analysis",  
    model="distilbert-base-uncased-finetuned-sst-2-  
english"  
)
```

The first thing I did was test how the model handles typos. I decided that five examples would be sufficient. The typo test set included variations of correct sentences with common spelling errors. The test results are presented in Table 1.

Table 1. Typo testing

No	Input	Output	Assessment
1	I love this movie	POSITIVE	correct
2	I loooove this movie	POSITIVE	correct
3	I lvoe this movie	NEGATIVE	incorrect
4	I lovee this moovie	POSITIVE	correct
5	Ths movie is gr8	NEGATIVE	incorrect

The model generally recognized the sentiment correctly. While double letters did not affect the score, but other types of typos – such as letter displacement (e.g., in example №3, where "o" and "v" are switched in the word "love" as "lvoe") – caused the confidence score to decrease, indicating sensitivity to orthographic noise. Naturally, a more complex form, like the phrase "Ths movie is gr8" is encrypted as "The movie is good", gave a false result.

Next was testing on slang (see Table 2).

Table 2. Slang testing

No	Input	Meaning	Output	Assessment
1	This movie slaps	The movie is excellent or high quality	NEGATIVE	incorrect
2	This movie is fire	The movie is amazing	POSITIVE	correct
3	That was meh	It was mediocre or boring	POSITIVE	incorrect
4	This movie sucks	The movie is very bad	NEGATIVE	correct
5	This show is lit	The show is exciting or amazing	POSITIVE	correct

The slang test shows that the model does not consistently understand informal language. While clearly negative slang like "This movie sucks" was classified correctly, some modern positive expressions such as "This movie slaps" were misclassified as negative. The term "meh", which usually indicates something mediocre or slightly negative, was predicted as positive. These results suggest that the model

12 February 2026

was trained mainly on standard movie review language and does not reliably interpret contemporary slang. This limitation is important in real-world applications where informal language is common.

On the last one I tested for irony (see Table 3).

Table 3. Irony testing

No	Input	True meaning	Output	Assessment
1	Oh great, another Monday. Just what I needed.	The speaker is unhappy about the start of the work week and feels overwhelmed.	POSITIVE	incorrect
2	Fantastic service... not!	The service provided was actually very poor and disappointing.	POSITIVE	incorrect
3	I'm so happy I failed my exam	The speaker is actually devastated or frustrated by their academic failure.	POSITIVE	incorrect
4	Yeah, that was the best meal ever... not.	The food was of very low quality and the speaker did not enjoy it at all.	POSITIVE	incorrect
5	I absolutely love waiting in traffic for hours	The speaker finds being stuck in traffic extremely annoying and a waste of time.	POSITIVE	incorrect

All ironical statements were misclassified. As in the slang test, the model takes the word in the sentence and its meaning literally, rather than being trained to understand the entire sentence. Most likely, it only catches "great", "fantastic", "happy", "best" and so on, and makes a conclusion based on that.

Conclusions

The testing of the distilbert-base-uncased-finetuned-sst-2-english model reveals significant gaps between laboratory performance and real-world application. While efficient, the model's reliance on clean training data makes it fragile when faced with the "messy" reality of human communication. The model shows some robustness to simple letter repetitions (e.g., "loooove"), but suffers from critical loss of accuracy when permuting letters or using numeric substitutes (e.g., "gr8"). This indicates that the model's tokenizer relies heavily on standard word spelling. The model fails to capture modern linguistic shifts, often interpreting positive slang like "slaps" as negative. Every ironical statement was misclassified because the model literally interprets positive keywords while ignoring the broader context or negations. The model tends to interpret words literally.

R1 Maksym Kotlubaiev
R2 A2406528
R3
R4
R5
R6

Learning Diary
R504TL194 Cloud Computing

4 (4)

12 February 2026

As the article "When data is messy" points out, real-world data is rarely clean. In real-world ML systems, these limitations are critical. Using such models in social media monitoring or customer support systems without additional fine-tuning on specific data will lead to erroneous business decisions due to misinterpretation of sarcasm or live user speech. Understanding these boundaries is essential for building robust, context-aware AI.

Sources

Jik, L. 2020. DistilBERT base uncased finetuned SST-2. HuggingFace Hub. Updated 2024. Accessed 12.2.2026 <https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>

Shane, J. 2020. When data is messy. AI Weirdness. Accessed 12.2.2026 <https://www.aiweirdness.com/when-data-is-messy-20-07-03/>.



Maksym Kotlubaiev